IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Mitigating Class Imbalance in Pump-and-Dump Detection: A Comparative Analysis of Imbalance-Aware Algorithms

M. MIRONOV[1], D. BOGUTSKY[2], and P. LUKIANCHENKO[3]

[1]National Research University Higher School of Economics, Faculty of Computer Science, Laboratory of AI in Mathematical Finance (e-mail: mvmironov@hse.ru)
[2]National Research University Higher School of Economics, Faculty of Computer Science, Laboratory of AI in Mathematical Finance (e-mail: dbogucki@hse.ru)
[3]National Research University Higher School of Economics, Faculty of Computer Science, Laboratory of AI in Mathematical Finance (e-mail: plukyanchenko@hse.ru)

Corresponding author: D. Bogutsky (e-mail: dbogucki@hse.ru).

This work was supported by the grant for research centers in the field of AI provided by the Ministry of Economic Development of the Russian Federation in accordance with the agreement 000000C313925P4E0002 and the agreement with HSE University № 139-15-2025-009

ABSTRACT The cryptocurrency market has been subject to various forms of exploitation, with pump-and-dump schemes being a persistent problem that can lead to significant price volatility and financial losses. Previous studies have focused on predicting attack targets using machine learning approaches; however, these methods are often compromised by class imbalance and biases inherent in historical data. To address this limitation, we present a novel framework for predicting manipulation targets. We employ two-step bias-minimizing data normalization techniques to mitigate the effects of class imbalance and temporal heterogeneity. We also propose several innovative machine learning approaches including imbalance-aware hyperparameters optimization and ranking algorithms. Our evaluation reveals that the proposed framework outperforms existing methods, achieving an accuracy rate of 19.0% and a top-10 accuracy rate of 56.9% in a highly imbalanced dataset of historical pump and dump events (imbalance ratio $\approx$ 290). For further validation, we construct a portfolio strategy based on the probabilities of predicted target classes to assess the practical application of our constructed models under realistic, slippage-aware execution.

INDEX TERMS Classification, Imbalanced Datasets, Learning to rank, Machine Learning, Market Manipulation, Market Microstructure, Portfolio strategy

I. INTRODUCTION

A. WHAT ARE PUMPS AND DUMPS

PUMP-AND-DUMP schemes (P&Ds) are a type of fraudulent activity that involves artificially inflating the price of a cryptocurrency through coordinated buying efforts, followed by a rapid selling to unsuspecting investors. This phenomenon is often characterized by an abrupt and significant increase in trading volume, followed by a subsequent decline in price as the original buyers liquidate their positions.

These schemes typically involve experienced traders and market manipulators who orchestrate the operation through various channels, including online forums and social media platforms such as Telegram channels. The manipulation is designed to create a false sense of urgency and scarcity, which attracts inexperienced investors who may be unaware of the true nature of the event.

A typical pump-and-dump scenario can be observed in Fig. 1, which shows a clear example of this phenomenon. The figure illustrates price movements associated with a specific token VIB traded against BTC on Binance exchange, highlighting the abrupt spike in trading activity following the announcement of the token's release. This sudden increase in volume is often followed by a rapid decline in price as insiders sell their positions to new buyers.

B. RELATED WORK

The phenomenon of pump-and-dump schemes has been extensively studied in the context of cryptocurrencies, with research efforts spanning multiple approaches and methodologies.

One of the pioneering studies on this topic was [9], which introduced a comprehensive framework for understanding pump-and-dump operations. The authors highlighted the tar-

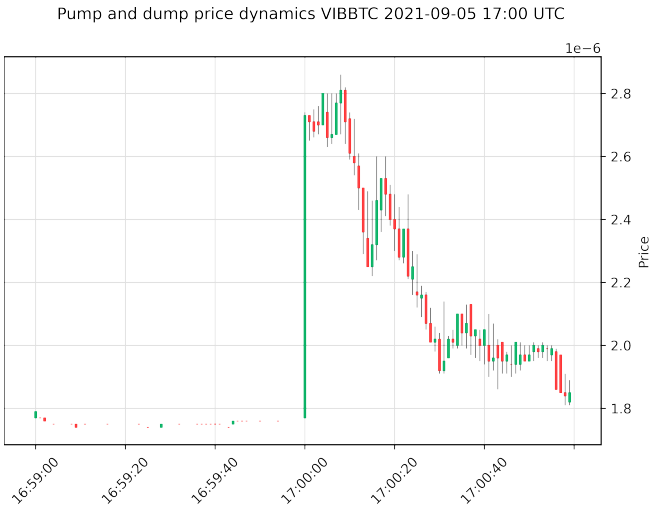


FIGURE 1. VIBBTC pump and dump manipulation carried out on Binance

geting of small market cap cryptocurrencies due to their lower trading volumes, utilizing data from 5 prominent crypto exchanges: Binance, Bittrex, Kraken, Kucoin, and Lbank. Their analysis covered pumps history of 1.5 years, with price and volume data being used as the primary sources. Subsequent research efforts have focused on leveraging machine learning (ML) algorithms to enhance P&D detection accuracy. La Morgia et al.'s work [10], [11] employed random forest algorithms, achieving improved results compared to traditional rule-based approaches. The authors in [4] further increased the effectiveness of La Morgia's work by incorporating Deep Neural Networks (LSTMs and Transformers) as anomaly detection algorithms. More recently, in [6] the authors built on these findings by studying a similar pump dataset between 2021 and 2022. They developed an innovative approach that employed resampling methods, such as SMOTE, combined with price and volume features and random forest classification to detect pump events 1 hour in advance.

In addition to pump detection, other research has focused on predicting the maximum price move during known pump events. Nghiem et al. [12] used market data and social sentiment with deep neural networks (LSTMs and CNNs) and achieved promising results. However, their analysis highlighted a potential limitation of using sentiment signals: historical bias may be introduced due to the lack of precise snapshots of historical social data.

A relevant example of work addressing the problem of predicting price manipulation targets is the study [17]. This research proposed an approach using Random Forest algorithms to identify potential pump and dump schemes among listed tokens. The authors analyzed a dataset of 180 pump events from the Cryptopia exchange, incorporating features such as price and volume data, global market capitalization data from Coinmarketcap, and token ratings by exchange. To enhance model robustness, the study employed filtering techniques to exclude pumps with suspicious announcement times that deviated significantly from full hours. Moreover, the authors validated their classification model through backtesting a portfolio strategy based on predicted pump targets. However, it is worth noting that the employed trading strategy relied on unrealistic assumptions, such as guaranteed profits of half the maximum price move during the pump and zero losses for false positive predictions.

A related study was conducted in [8], where authors enhanced their approach by incorporating channel embeddings into their model. They also utilized standard price and volume metrics, as well as external data sources such as market capitalization and social media information (e.g., Twitter followers, Reddit subscribers, Alexa rank). The analysis was performed on a dataset comprising 445 pump announcement events. However, due to the presence of duplicate pump events across multiple channels, the actual number of unique pump events studied is smaller than expected. Furthermore, the use of social data from different sources raises concerns about historical bias, as these metrics may have changed after the fact, potentially affecting the accuracy of the results.

Regarding market manipulation in a broad sense, several papers have also been published recently, for example [5], which studied the manipulation of trading volumes, also known as wash trading.

With the increasing popularity of ML applications in finance, in recent years we have seen many articles on predictions using the market microstructure, for example [2] and [13]. Promising results have also been published with applications of ranking models to finance [14]. Taking into account the rich accumulated experience in related areas, we aim to complement existing work with new factors based on the market microstructure.

In the broader ML literature, several families of techniques have been proposed for handling class imbalance beyond resampling, and each maps imperfectly onto the P&D target-prediction problem. Cost-sensitive learning [25] re-weights the loss so that misclassifying the minority class is more expensive; it does so without perturbing the feature space, which, as we show empirically in Section IV, is a decisive advantage when features are already cross-sectionally normalized. Focal loss [27], originally proposed for dense object detection, down-weights easy examples and concentrates the gradient on hard, informative ones; it has since been adopted for imbalanced tabular classification, but remains most effective when combined with deep models and large minority classes, conditions that do not apply to our 227 positive training events. Synthetic oversampling methods such as SMOTE [18] have known pathologies in high-dimensional settings [22], which we confirm in Section IV.

Anomaly detection provides a complementary formulation that does not require balanced training data. Isolation Forest [28] and related one-class approaches have been applied to fraud and market-abuse detection when positive examples are essentially unavailable. In our setting, however, positive examples *are* available (at least in limited numbers) and the task is explicitly *cross-sectional*: we do not ask "is this

observation anomalous in isolation?'' but rather ''which asset in this cross-section is the most likely manipulation target?''. This ranking-within-cross-section structure makes the problem closer to learning-to-rank than to unsupervised anomaly detection, and motivates the imbalance-aware ranking metric (Top@K%-AUC) that we adopt as the training target throughout this paper. To the best of our knowledge, no prior work on P&D target prediction has combined cross-sectional panel normalization, microstructure features, and an imbalance-aware ranking objective explicitly tailored to the 1-positive-vs.-many-negatives structure of each cross-section.

This paper builds upon previous work in [17] and [8], which focused on predicting manipulation targets among traded assets. We identify several critical limitations in data preparation that limit model reliability and propose innovative solutions to address these issues. Specifically, we introduce novel microstructure-based features leveraging insights from areas of market prediction [2], [13] and wash trading detection([5]). Moreover, we explore the use of boosted trees, as implemented in CatBoost [15], which has been shown to outperform other algorithms on tabular data (e.g., [7]), and we compare their performance to the Random Forest model introduced in [3].

To further improve the classification models' performance, we also perform a cross-sectional normalization method for panel data. Moreover, we formulate a specialized metric tailored to optimize the classification model for highly unbalanced datasets, which was used for model training and validation.

Table I summarizes the key methodological differences between our framework and the most closely related prior studies.

II. DATA COLLECTION AND PREPROCESSING

A. LABELED P&DS DATA

To investigate pump-and-dump schemes, we collected a dataset of P&D events from various sources. Initially, we downloaded chat history from 8 Telegram channels that were known to be associated with these types of activities. We manually labeled each event, extracting relevant information such as the ticker symbol, pump time, and exchange platform. Our sample spans the period from December 18, 2018 to April 1, 2024, covering a total of 175 P&D events. The majority of Telegram channels used for organizing P&Ds have since become inactive, resulting in their chat histories being deleted by Telegram. Consequently, our dataset is smaller than those obtained in previous studies. To supplement our sample and increase its diversity, we incorporated the labeled pumps and dumps dataset from La Morgia et al. (2020) [10], which they had updated in 2021. This additional dataset comprises 1111 P&D events across multiple exchanges.

a: Channel identification.

We identified eight candidate Telegram channels through a two-step procedure. First, we searched Telegram directly using the keywords ''crypto pump,'' ''pump signal,'' ''pump and dump,'' and their localized equivalents, retaining public channels whose stated purpose was the coordination of pump events. Second, we cross-referenced the resulting list with channels named or cited in prior literature [6], [10], [17]. For each channel, we exported the full message history (JSON format) using the Telegram Desktop client's built-in export functionality. The channel names cannot always be redistributed, but the timestamps and tickers of the retained events are in the public event file.

b: Labeling procedure.

One annotator labeled candidate events by inspecting every message that contained a pump-like announcement, using a fixed template (ticker, exchange, announcement time, message URL). Ambiguous messages (multiple tickers, corrections, re-posts) were resolved by keeping only the first timestamp per {channel, ticker, day} triple; when the same pump was announced in several channels, the earliest announcement time was retained. The complete set of labeled candidates (including those later dropped) is stored in `resources/pumps.json` so a second annotator can replicate or audit the decisions.

c: Inclusion criteria.

A candidate event was retained as a valid pump only if *all* of the following were satisfied:

- **Ticker identification:** the message explicitly names a ticker symbol and identifies the target exchange.
- **Timestamp extraction:** the announcement time is the Telegram message timestamp in UTC, rounded to the nearest second.
- **Price-volume verification:** within the first 5 minutes after the announcement, the BTC-quoted price of the ticker increased by at least 5% and the traded quote volume exceeded $3\times$ the preceding 60-minute rolling average. Verification is done against the 1-second resolution trade history downloaded from Binance's public historical archive (https://data.binance.vision).
- **Announcement regularity:** the announcement timestamp falls within ± 5 minutes of a full hour or half hour. Off-grid announcements often indicate pre-leakage or late-corrected posts and are excluded, following [17].

d: Exclusion criteria.

Candidate events were dropped if any of the following held: (a) the ticker was not listed on Binance against BTC at the announcement time; (b) the price-volume response failed either of the quantitative thresholds above (failed or postponed pumps); (c) the announcement timestamp was off-grid (out of the ± 5-minute window); (d) the event was a duplicate of an earlier kept event (same ticker and exchange, announcement time within 5 minutes); (e) the target exchange was not Binance, since we restrict microstructure analysis to a single venue for comparability. Dropped events with their rejection reasons are preserved in `resources/dropped_pumps.json`.

TABLE I. Comparison of the proposed framework with closely related prior work on P&D target prediction

Study	Exchange	# Events	Model	Imbalance Handling	Backtesting
Xu & Livshits [17]	Cryptopia	180	Random Forest	None	Unrealistic assumptions
Hu et al. [8]	Multiple	445	Seq. model + channel embed.	None	None
Fantazzini & Xiao [6]	Binance	Varies	Random Forest	SMOTE	None
La Morgia et al. [10]	Multiple	1111	Random Forest	None	None
Ours	Binance	359	CatBoost, LR, RF, Ranker	Cross-sectional norm., Top@K%-AUC opt., class weighting	Impact-aware execution

We performed the following preprocessing steps to prepare this combined dataset:

1) *Duplicate removal*: We filtered out any duplicate events that were present in both datasets.
2) *Criteria-based filtering*: We removed events that did not meet our criteria for a pump, specifically those with minimal changes in price and volume during the event and non-regular time of the announcement. These instances may indicate failed or postponed pumps, which we deemed irrelevant to our analysis.
3) *Exchange filtering*: We retained only P&D events organized on Binance from both datasets.

The resulting dataset contains 359 unique market manipulation events. Restricting the analysis to Binance BTC pairs is a deliberate design choice with known trade-offs: (i) we lose events that targeted USDT-quoted pairs, tokens listed only on smaller venues (e.g., KuCoin, Gate, LBank, Cryptopia before its 2019 shutdown), and decentralized-exchange pumps; (ii) in exchange, we obtain full trade-level history for every pair in the cross-section, which is the minimum requirement for computing slippage, flow-imbalance and power-law features consistently across the whole sample; (iii) the restriction is aligned with the most comparable prior work on Binance P&D target prediction [6] and on microstructure-based detection [10], so our results remain directly comparable. We further discuss the limits of external validity in Section VI. Our data presents a substantial challenge, with P&D events occurring relatively infrequently compared to normal trading activity. During our study period, the average number of instruments traded (193.7) represents nearly all listed pairs on Binance with BTC as the quote asset, excluding the most liquid pair ETHBTC. The historical dynamics of the number of listings is shown in Fig. 2.

We split the data into training, validation and testing samples by 2020-09-01 and 2021-05-01 respectively. Table II describes the resulting datasets after the split:

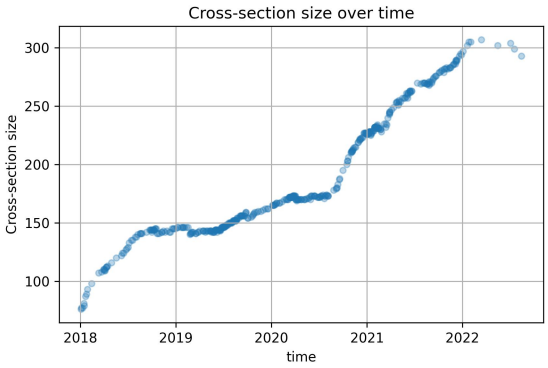


FIGURE 2. Binance has been increasing the number of tickers traded against BTC by listing more tokens. This tendency was likely to remain up until the end 2022 when more regulation came in with much stricter listing requirements.

	Train	Validation	Test
Positive	227	74	58
Negative	34888	17399	16892
Total	35115	17473	16950
Avg. Cross-section Size	154.69	236.12	292.24

TABLE II. P&D Train-Validation-Test samples statistics

B. MARKET DATA

To enable comparison with existing research, we selected Binance as the sole exchange platform for our analysis. We retrieved trade-level data for all tickers traded on the platform from Binance API up to March 31, 2024. Given that our collected pump events exclusively involved pairs with BTC as a quote currency, we limited our analysis to such pairs, thereby ensuring consistency with existing studies in this area.

A representative example of our collected data is presented in Table III.

	price	qty	time	isBuyerMaker	quote
0	0.000002	2437.00	2022-07-15 15:00:02.422000	True	0.004240
1	0.000002	275.00	2022-07-15 15:00:02.434000	False	0.000479
2	0.000002	1125.00	2022-07-15 15:00:22.884000	True	0.001946
3	0.000002	941.00	2022-07-15 15:00:52.277000	True	0.001628
4	0.000002	184.00	2022-07-15 15:00:52.277000	True	0.000318

TABLE III. Structure of collected data: *price* shows the price that the order was filled at, *qty* is the amount of base asset traded, *isBuyerMaker* indicates if the buyer's order related to this transaction was already in the orderbook: if True, the trade is classified as a Sell event and if False the trade is characterized as a Buy event.

C. FEATURE ENGINEERING

We collected trade-level data as compressed monthly archives for each ticker traded on Binance through March 31, 2024. Because trading activity varies substantially across tickers and over time, the raw event streams are irregular and cannot be used directly as fixed-size inputs to standard machine-learning models. We therefore implemented a feature-generation pipeline that (i) decompresses and loads the relevant archives, (ii) partitions data into time-aligned windows, (iii) computes microstructure features summarizing pre-announcement order flow, and (iv) aggregates ticker-level features into an event-level cross-section associated with each pump. The pipeline is:

1) Download trade-level data from the Binance API.
2) For each pump event, construct a cross-section of tickers traded over the same time range.
3) For each ticker in the cross-section, load up to 30 days of history prior to the announcement and compute features over predefined lookback windows ending one hour before the event.
4) Combine features for all tickers into a single event-level dataset and assign labels by adding *is_pumped*, which equals 1 for the pumped ticker and 0 otherwise.

This modular pipeline allowed fast iteration over alternative feature sets. Implementing it efficiently required substantial effort due to the scale of the data; after multiple optimizations, feature generation time decreased from several days to approximately two hours. The implementation and full instructions to reproduce our results are available in [19]. We group features into several categories, described below.

1) Imbalance features

Volume Imbalance is measuring which side is more prevalent in trade data and equals ratio of signed volume in quote asset (negative for sell orders, positive for buy orders) over absolute volume in quote asset. For instance, volume imbalance measure skew towards buying or selling. If imbalance ratio is closer to 1, this implies that there are more buyers than sellers and vice versa.

$$\text{Volume Imbalance} = \frac{\sum_{t\in T} qty^t_{signed}}{\sum_{t\in T} |qty^t_{signed}|}, \tag{1}$$

where qty^t_{signed} - signed volume of trade at time t with sign depending of the side of the trade: +1 for buy, -1 for sell.

2) Quote slippages

We also created features based on slippage. Since we do not have information about order types, we cannot determine whether a trade was executed via a market or limit order; therefore, we treat slippage as a useful proxy for measuring market participants' impatience. We follow the methodology used in prior work by grouping ticks by execution time. Given that we deal with highly illiquid markets—where the time between trades can be as long as 15 minutes—we can reasonably assume that aggregated ticks correspond to a single order. This allows us to estimate the price impact of the aggregated order. The intuition is as follows: if a market participant is willing to incur large slippage losses by submitting a large buy order, they likely want to build their position quickly, potentially in anticipation of the pump. Based on this idea, we compute slippage for each aggregated trade using the formula in (2) below:

$$\text{Slippage Loss} = \underbrace{\sum_{i=1}^{N} qty^i_{signed} \cdot P_i}_{\text{Quote actually spent}} - \underbrace{\sum_{i=1}^{N} qty^i_{signed} \cdot P_0}_{\substack{\text{Quote could have been spent} \\ \text{if filled at best price}}}, \tag{2}$$

where qty^i_{signed} is the signed quantity of the i^{th} order within an aggregated trade consisting of N orders. This formula defines slippage loss as the difference between the quote amount actually spent and the quote amount that would have been spent if the entire trade had been executed at the best bid or ask price (depending on the trade side). As an order is executed, it consumes liquidity from the order book, which appears as a sequence of prints in trade-level data. This metric is intended to capture how deeply the order consumes the order book.

3) Log returns features

We have also followed methodology of [17] and calculated features using a window of sizes (x+1, 1) where x is different offset in hours or days. Left bound indicates the start of the window, right bound of 1 indicates that we calculate features up to 1 hour prior to the announcement of the pump. This way we calculated mean and standard deviation of hourly log returns using windows described above. We hope that such approach could catch some anomalous hourly log returns which could point at insider buying before the pump. We

calculated time-series normalization for returns (z-scores) using long term mean and average.

4) Power law features. Quantile spread of order sizes

We also studied sizes of trades, we mostly were interested in big volume trades executed before the pump. In order to measure this we applied Power Law stating that data in higher quantiles has the following (3) probability density function:

$$f(x, \alpha) = \alpha x^{\alpha - 1} \tag{3}$$

Using this we estimated a value of $\hat{\alpha}_{\text{PL}}$ using samples of various sizes following the same idea of windows as described in the previous section. Since α is in the exponent of $f(x, \alpha)_{\text{pdf}}$, therefore lower values of alpha result in PDF decreasing slower which is a sign of heavier tails. This implies that there is a higher spread in upper quantiles showing the presence of abnormally big trades. In order to estimate alpha, we first filtered out all data below 95% quantile and estimated sample probability density function (PDF). Then, we regressed logarithm of estimated PDF against logarithm of *quote_abs* feature:

$$\log \hat{f}_{\text{pdf}} = \underbrace{\log \alpha}_{\text{intercept}} + \underbrace{(\alpha - 1)}_{\text{slope}} \log \left(\text{quote\_abs}\right) \tag{4}$$

As a result of this procedure we took estimated slope which is $\alpha - 1$ and added back 1 to get $\hat{\alpha}_{\text{OLS}}$, then we used this estimate as a feature measuring spread in order sizes in upper quantiles.

All of the features discussed are further explained below (see Table IV):

D. CROSS-SECTIONAL APPROACH

Given that our dataset spans a significant period from early 2018 to 2024, we account for potential temporal heterogeneity by applying cross-sectional standardization. This approach enables us to remove market regime changes and focus on the underlying cross asset dynamics.

We performed cross-sectional standardization by subtracting the mean value of each numerical feature within a given event from the original values. We then divided these adjusted values by their respective standard deviations, calculated in the same manner:

$$X_{\text{standardized}} = \frac{X - \bar{X}_{CS}}{\bar{\sigma}_{CS}} \tag{5}$$

Note that this standardization was not applied to categorical features or to features with bounded values, such as various imbalance ratios. Fig. 3 demonstrates the impact of the normalization technique on a sample of eight cross-sections for three numerical features:

III. MODELLING

In this section, we will attempt to create a model which is able to label this fraudulent activity 1 hour before the announcement. As discussed earlier this is a challenging task due to high imbalance in the data. The task of finding manipulated assets can be designed as a binary classification problem where "1" implies that the asset is pumped and "0" that it is not, just as was described earlier. Therefore, we trained different popular classification models like LogisticRegression, RandomForest and CatboostClassifier to solve this task. But also this problem can be designed differently: it is reasonable to assume that the asset which is manipulated should significantly increase in price after the announcement and therefore have one of the highest returns among all assets traded. Following this idea we ranked assets based on their maximum return within the first 5 minutes after announcement and ranked all assets based on this maximum return. The idea is that we can train ranking model which is able to order assets based on the microstructural features described earlier and hence predict which asset is going to be manipulated as it should be among the ones ranked the highest. In order to implement this approach, we used CatboostRanker and then we compared such approach to classification models.

A. EVALUATION METRICS

As one of the evaluation metrics we used is Top@K accuracy which is defined as a probability that among K highest logits of the model there will be the one that is the ground truth. In our context, if we compose a portfolio of size K based on logits of our models, we will catch pump with Top@K probability. This metric is useful as it allows us to estimate the performance of portfolios which we will cover later. We also compared the models based on Top@K% accuracy for different values of percentage threshold K%. This is a modification of the regular Top@K allowing us to control for the problem of increasing cross-section sizes described in Section (II-A). And the final and most important metric that we used is Top@K%-AUC which is the area under the Top@K% curve.

The area under the Top@K% curve (Top@K%-AUC) serves as our primary optimization target. In our setup we integrate the Top@K% curve over the low-$K\%$ region $K\% \in (0,\ 20\%]$ and normalize by the length of the integration range so the metric remains in $(0, 1)$:

$$\text{Top@K\%-AUC} \ = \ \frac{1}{0.20} \int_0^{0.20} \text{Top@K\%}\, d(K\%). \tag{6}$$

Restricting integration to the $(0, 20\%]$ region is motivated by two considerations. First, it is the economically relevant region for a concentrated P&D portfolio: a practical strategy holds at most the top 10–20% of the cross-section in any given event. Second, all reasonably-trained models saturate above $K\% \ \approx \ 30\%$ and their Top@K% curves converge, so integrating over the full $(0, 1)$ range dilutes the metric with a large K region where every model is close to 1 and no longer discriminates between them. Truncating at 20%

Feature	Description	Notation
Asset return	return of the asset calculated over the window of size (T-1H-x, T-1H) prior to the pump announcement	*asset_return*[x]@*t*
Standardized hourly return	average hourly return in quote asset (BTC) calculated over the window of size (T-1H-x, T-1H) and divided by standard deviation of hourly returns calculated over the last 30 days	*asset_return_zscore*[x]@*t*
Scaled average volume in BTC	the same as above but everything is calculated using only hourly trade volumes	*quote_abs_zscore*[x]@*h*
Share of long trades	share of long trades as the ratio of number of long trades and all trades of this time window (T-1H-x, T-1H)	*share_of_long_trades*[x]@*h*
Share of long trades	share of long trades within window (T-1H-x, T-1H)	*share_of_long_trades*[x]@*h*
Flow imbalance ratio	volume imbalance ratio computed over the window of size (T-1H-x, T-1H)	*flow_imbalance*[x]@*h*
Quote slippage imbalance ratio	imbalance ratio of slippages in BTC calculated over the same time window	*slippage_imbalance*[x]@*t*
Powerlaw alpha	estimated alpha from powerlaw probability density function using the window of size (T-1H-x, T-1H) prior to the pump	*powerlaw_alpha*[x]@*h*
Number of previous pumps	number of times this token has been pumped before within the last 90 days	*num_prev_pumps*

TABLE IV. These are the features included in all models. These features are used to describe historical dynamics of price and trading volume in the run up to the pump. $x \in \{5m, 15m, 1h, 2h, 4h, 12h, 1d, 2d, 7d, 14d\}$

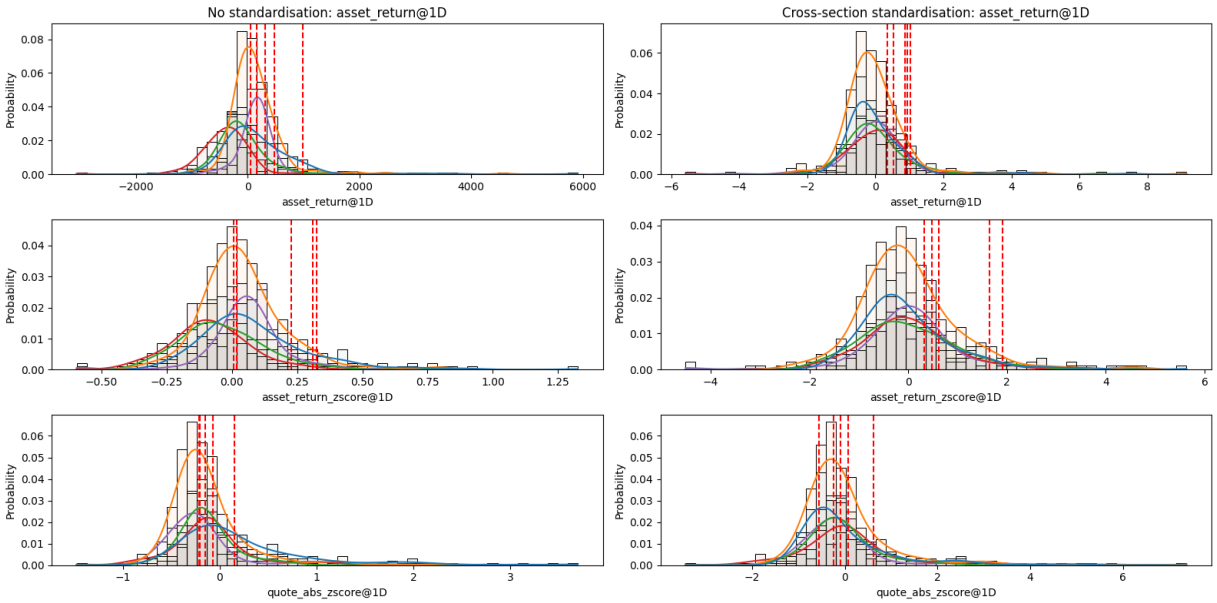


FIGURE 3. Features without and with cross-sectional standardization: We plotted histograms for 8 different P&D events and also added KDE of PDF. Applying standardization helps to align distributions more closely with one another across pumps.

makes Top@K%-AUC materially more sensitive to differences between models and to hyperparameter choices, which is the property we want from both an Optuna tuning target and an early-stopping signal inside CatBoost. Numerically, Top@K%-AUC values reported in this paper are therefore not directly comparable to AUCs integrated over $(0, 1)$; a model that saturates quickly below 20% can attain a Top@K%-AUC close to 0.8 under this convention.

1) Limitations of Top@K-based metrics

While Top@K metrics are directly relevant for portfolio construction (how many assets must we hold to catch the pump?), they have limitations. They do not measure overall classification quality across the full score distribution and can

mask poor calibration. A model may achieve high Top@K by ranking the pump target above most non-targets without producing well-separated probabilities. To address this, we complement Top@K with standard classification metrics.

2) Standard classification metrics

We additionally report three standard metrics computed on the full test set:

- **PR-AUC:** Area under the precision–recall curve, which is more informative than receiver operating characteristic (ROC) AUC under extreme class imbalance [25].
- **F1-score:** Harmonic mean of precision and recall, computed using a "top-1 per cross-section" decision rule (predicting the highest-scored asset as positive).
- **Balanced accuracy:** Average of sensitivity and specificity, providing a class-imbalance-aware accuracy measure.

B. BASELINE MODEL

We chose Logistic Regression - the simplest model as a baseline model to see if it outperforms RandomClassifier which chooses which ticker is going to be manipulated randomly. By doing this we wanted to check if extracted features have any actual predictive power about future pumps. We trained the LogisticRegression model as is, the only thing we did was adjust *class_weight* of the minority (positive) class to penalize more heavily for pump mislabeled. We trained the model using the entire train sample and were able to obtain good enough results, presented in Table V. Our primary evaluation metrics are Top@K with values of $K \in \{1, 2, 5, 10, 20, 30\}$ and Top@K%-AUC. We will come back to these metrics in more details in Section IV.

C. TRAINING

We used the train sample to train the models. In order to control overfitting we tried to use shallow trees by limiting *max_depth* parameter in both RandomForest and Catboost models. We also applied early stopping for both CatboostClassifier and CatboostRanker and used validation set to compute evaluation metrics. We created a custom evaluation metric Top@K%-AUC , this metric was used by the trainer to find the optimal number of boosted trees. We also set the number of early stopping rounds to 50 which implies If there is no improvement in Top@K%-AUC for more than 50 rounds, the training process will stop and the model will rollback to the best state. This was used in the training process of the model CatboostClassifier + TOPKAUC Early stopping (early stopping model with custom evaluation metric).

We also tuned hyperparameters for all of the models using the separate validation set which we made sure is large enough to be more or less representative of the train sample. Then, we used *Optuna* package [1] which implements Bayesian optimization and allows to go through different sets of hyperparameters more efficiently as compared to simple grid search. Each model was trained 30 times and evaluated on the validation sample, the set of parameters producing the best results in terms of the chosen TOPKAUC metric was chosen and used to train the model.

We also used SMOTE proposed by [18] to tackle the problem of class imbalance. It allows to create new observations for the minority class to balance the dataset. Instead of copying existing minority points, it generates new, slightly different ones by interpolating between neighbors.

IV. RESULTS

A. TOP@K ACCURACY

Following the methodology described above, we trained various models with the best hyperparameters found by *Optuna* package. Then, we computed Top@K accuracy for each model on the test set. The results are presented in Table V.

We can see from the table that baseline LogisticRegression is holding up very well compared to other more complex models like RandomForest and CatboostClassifier. The superior model turned out to be CatboostClassifier + TOPKAUC Early Stopping (abbreviated Catboost + ES), the model was trained using early stopping with a custom evaluation metric - Top@K%-AUC which measures the performance of the model for all values of $K\%$. It outperforms all models across almost all values of K. We also see that SMOTE did not solve the problem of class imbalance in our case, it has very poor performance for all values of K being even much worse than simple LogisticRegression. Therefore, adjusting weights in the binary logloss is a preferred way to tackle the problem of imbalance in the target variable. Similar comparison of the models in terms of Top@K accuracy is shown below in Fig. 4:

As we can see, all models surpass the performance of RandomModel which randomly chooses the next pumped asset. Top@K values for the RandomModel are computed as follows:

$$\text{Top@K}_{\text{random}} = \frac{1}{N} \sum_{i=1}^{N} \frac{K}{T_i} \tag{7}$$

where N is the number of cross-sections in the test set, T_i - i-th cross-section size. This implies that our models are able to consistently capture underlying patterns in the data that describe P&D events. Ranking models seem to be underperforming when compared to classification models. It is only able to compete with others in terms of Top@1 accuracy, but its performance at other values of K is much worse than others.

B. TOP@K% ACCURACY

Then, we estimated Top@K% for all models using the same test sample, the results are shown below in Table VI:

We see that once again CatboostClassifier + TOPKAUC Early Stopping is the best in terms of Top@K% for almost all values of $K\%$. Only at 20% threshold it slightly loses out to RandomForestClassifier.

TABLE V. TOP-K accuracy of models on test sample

Model	TOP-1	TOP-2	TOP-5	TOP-10	TOP-20	TOP-30
LogisticRegression	0.172	0.224	**0.379**	0.483	0.672	0.707
LogisticRegression + Tuned	0.155	0.190	0.345	0.552	0.707	0.759
RandomForestClassifier	0.121	0.224	0.293	0.379	0.569	0.707
RandomForestClassifier + Tuned	0.155	**0.259**	0.362	0.517	0.655	0.793
CatboostClassifier	0.172	0.241	**0.379**	0.483	0.552	0.690
CatboostClassifier + Tuned	**0.190**	0.241	**0.379**	0.534	0.707	0.793
CatboostClassifier + SMOTE	0.017	0.121	0.207	0.293	0.448	0.569
CatboostClassifier + SMOTE + Tuned	0.086	0.121	0.259	0.414	0.552	0.638
CatboostRanker	0.017	0.069	0.172	0.362	0.534	0.621
CatboostRanker + Tuned	0.086	0.138	0.190	0.397	0.517	0.569
CatboostClassifier + TOPKAUC Early Stopping	**0.190**	0.224	**0.379**	**0.569**	**0.724**	**0.810**

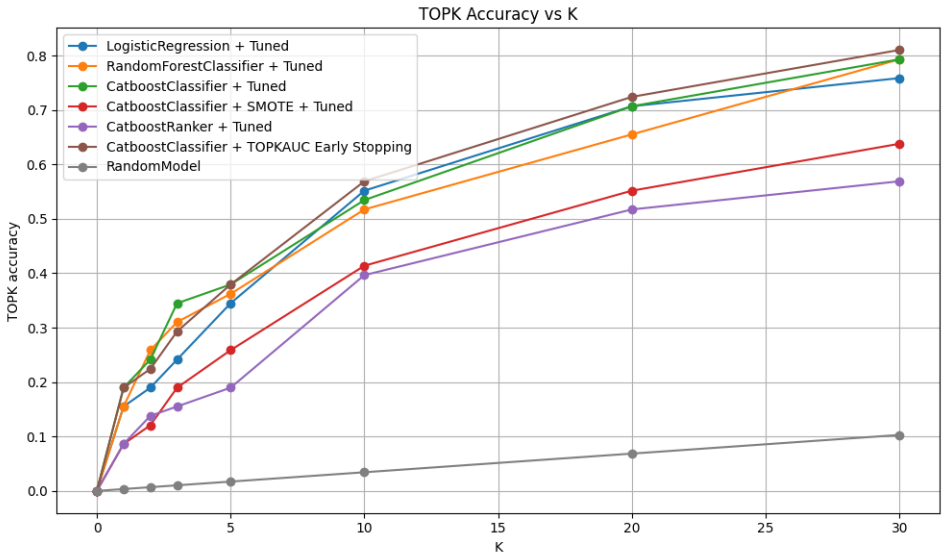


FIGURE 4. Top@K accuracy on the test sample for all models and the random-ranking baseline. The horizontal axis is the portfolio size *K*; the vertical axis is the empirical probability that the manipulation target appears among the *K* highest-scored assets.

TABLE VI. TOP-K% accuracy of models on test sample

Model	1%	2%	5%	10%	20%	50%
LogisticRegression	0.276	0.397	0.569	0.707	0.828	0.966
LogisticRegression + Tuned	0.241	**0.431**	**0.655**	0.759	0.879	**0.983**
RandomForestClassifier	0.276	0.310	0.483	0.707	0.828	0.914
RandomForestClassifier + Tuned	0.345	0.414	0.569	0.793	**0.914**	**0.983**
CatboostClassifier	0.328	0.379	0.500	0.690	0.862	0.966
CatboostClassifier + Tuned	**0.362**	0.414	**0.655**	0.793	**0.914**	0.966
CatboostClassifier + SMOTE	0.172	0.241	0.362	0.569	0.672	0.914
CatboostClassifier + SMOTE + Tuned	0.224	0.310	0.517	0.638	0.741	0.948
CatboostRanker	0.138	0.293	0.500	0.621	0.759	0.862
CatboostRanker + Tuned	0.155	0.293	0.448	0.569	0.759	0.862
CatboostClassifier + TOPKAUC Early Stopping	0.293	**0.431**	**0.655**	**0.810**	0.879	**0.983**

C. TOP@K%-AUC

In order to control for the problem of various cross-section sizes, we want to compute Top@K% instead of regular Top@K. Taking area under the Top@K% curve allows us to evaluate performance for all values of $K\% \in (0, 1)$. In Fig. 5 we compare the models based on this metric.

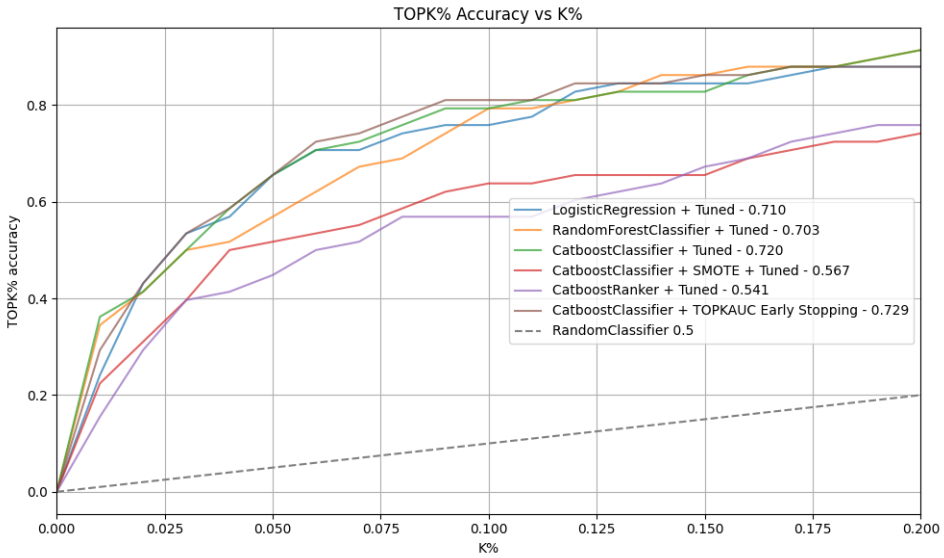


FIGURE 5. Top@K% accuracy on the test sample, normalizing the portfolio size by the cross-section size to control for its growth over time. The horizontal axis is the fraction $K\%$ of the cross-section retained; the vertical axis is the probability that the target is covered.

D. CLASSIFICATION METRICS

We complement the ranking-based Top@K metrics with three standard classification metrics (defined in Section III-A): PR-AUC, F1-score (top-1-per-cross-section decision rule), and balanced accuracy. PR-AUC is preferred over ROC-AUC at this imbalance ratio [25]. Fig. 6 reports all three for every tuned model and for Catboost + ES: the latter attains the highest PR-AUC (0.1036), and ties with CatboostClassifier + Tuned for F1 (0.1897) and balanced accuracy (0.5934). The two imbalance-aware CatBoost variants therefore dominate the standard classification measures as well as the ranking metrics, ruling out the possibility that the Top@K advantage comes from miscalibrated scores.

E. ANALYSIS OF SMOTE PERFORMANCE

SMOTE-based CatBoost variants performed substantially worse than the class-weighted classifiers. The untuned SMOTE model reached only 1.7% Top@1 accuracy, and even the tuned SMOTE variant reached just 8.6%, compared with 19.0% for both CatboostClassifier + Tuned and Catboost + ES. The gap is large and consistent across all Top@K thresholds, which rules out a tuning artefact. We attribute the failure to a structural mismatch between SMOTE's generative assumption (local linear interpolation between minority samples in feature space) and three specific properties of our panel microstructure features.

1) **Bounded and sign-meaningful features.** Several of our features—flow imbalance, slippage imbalance, share of buy-initiated trades—are naturally bounded in $[-1, 1]$ (or $[0, 1]$) and are informative primarily near the positive extreme (e.g., heavy buy-side imbalance just before a pump). Interpolating between two pumped samples whose imbalance is $+0.85$ and another whose imbalance is -0.20 yields a synthetic "pump" with imbalance $+0.32$, which lies inside the bulk of the negative-class distribution. SMOTE therefore dilutes rather than reinforces the class-discriminative region of the feature space.
2) **Cross-sectional reference.** Every numerical feature we feed to the classifier is standardized within the cross-section of tickers active at the pump time (Eq. 5). A feature value of $+2$ means "two cross-sectional standard deviations above the same-event average," and is meaningful only relative to the other 290-odd tickers trading that hour. SMOTE's nearest-neighbor interpolation mixes values from *different* cross-sections, producing synthetic samples whose feature values have no corresponding cross-section in which they are coherent. The synthetic samples are in-distribution marginally but out-of-distribution jointly with respect to the panel structure.
3) **High-dimensional sparsity.** With more than 70 features and only 227 positive training samples, the nearest-neighbor graph that SMOTE relies on is sparse and noisy: each minority point has few true neighbors in its class, and interpolation typically bridges across different sub-populations (e.g., small-cap vs. medium-cap pumps). This is the regime in which Blagus and Lusa [22] showed that SMOTE is at best neutral and frequently *degrades* classification performance.

By contrast, class-weight reweighting in the CatBoost loss function does not add, remove, or perturb any feature value. It only changes the per-sample gradient magnitude, concentrating the optimizer on the decision boundary between the real positive and negative samples in their native, cross-sectionally-normalized feature space. This explains the con-

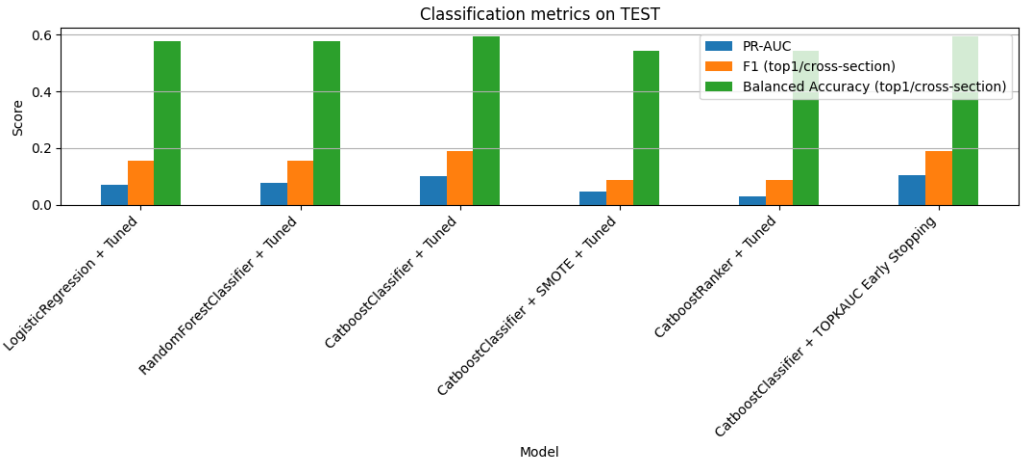


FIGURE 6. **Standard classification metrics on the test set for all tuned models and Catboost + ES. Each grouped bar reports Precision-Recall AUC (left), F1-score under a top-1-per-cross-section decision rule (middle), and balanced accuracy, the average of sensitivity and specificity (right). PR-AUC is preferred over ROC-AUC under the $\sim$ 1:290 class imbalance. Takeaway: the two imbalance-aware CatBoost variants dominate the standard classification measures as well as the ranking metrics, ruling out a calibration artefact.**

sistently superior performance of class-weight-based imbalance handling over synthetic oversampling in our setting and is consistent with the cost-sensitive-learning literature [25].

F. ANALYSIS OF RANKING MODEL PERFORMANCE

The CatboostRanker underperformed classification models across all metrics. Even the better of the two ranker variants reaches only 8.6% Top@1 accuracy (1.7% untuned), which is substantially lower than the baseline Logistic Regression (17.2%). A natural expectation from the learning-to-rank literature [14], where listwise ranking outperforms pointwise regression in cross-sectional equity alpha prediction, is that ranking should dominate here as well. That expectation does not hold in our setting, and the deviation is informative.

Target-signal density. In the cross-sectional LTR setup of [14], every asset in the cross-section carries an informative continuous return target, so the listwise loss receives dense, ordinally-meaningful supervision. In our setup, exactly one asset per cross-section is a true manipulation target. All other assets contribute noise to the ordering: their 5-minute post-announcement returns are driven by background market movements, independent news, or incidental volatility, none of which the model is asked to predict. The ranker therefore spends most of its gradient on nuisance ordering among non-pumped assets, while the classifier only has to separate the single pump from the rest.

Loss-function alignment with the Top@K evaluation objective. Pairwise and listwise ranking losses (YetiRank, PairLogit, etc.) integrate pairwise or position-weighted comparisons across the entire list. When the positive cardinality is 1, most of those pairs are negative-vs.-negative and therefore non-informative for Top@K. The classification log-loss, conversely, concentrates gradient mass on the decision boundary between the single positive and the $\sim$290 negatives, which is exactly the boundary that Top@K accuracy measures. Class-weight reweighting further tilts the log-loss toward the positive class, yielding a gradient that is sample-efficient in the 1-vs.-many regime.

Target-construction noise. The continuous ranking target (maximum 5-minute return after announcement) is additionally noisy because (i) coincident market-wide moves can push a non-pumped asset's max-5-minute return above the pump target's, and (ii) micro-cap order books produce large relative returns on very small trades, which the ranker has no way to discount. In the classification formulation the binary label is immune to this type of noise.

Taken together, these three factors explain why, in the extreme-imbalance cross-sectional setting that characterizes P&D target prediction, classification with class-weight reweighting dominates listwise ranking even though the evaluation metric is itself a ranking metric.

We can see that all models perform relatively well in terms of this metric, with Catboost + ES attaining the highest Top@K%-AUC (0.729). We mostly care about performance at low values of K, where this model is strongest.

G. INTERPRETATION OF THE MODELS

In this section we will move from comparison of the models and focus on interpretation of the obtained models. We will see which features had the most impact and try to explain why this makes sense in the context of prediction of P&Ds. We computed feature importance as a change of prediction value given the change in the value of feature; this way we are able to reason which features affect the target variable the most. As we can see from the bar graph below (see Fig. 7) *num_prev_pump* which is the number of times the token was pumped before another manipulation is among the three most impactful features. During manual data collection from Telegram, even then we noticed that it was often a case when manipulators pumped the same token 2 times in

a row or chose tokens that they already knew how to pump. This tendency seemed to persist across the whole dataset so that is why this is one of most important features. Then, we have *slippage_imbalance* features and *quote_abs_zscore* features. The first might indicate that insiders are buying into the manipulated asset before the pump using mostly market orders, therefore, since the orderbooks are mostly empty, they will be incurring slippage costs, this is why this feature is important as it describes low liquidity tokens which are bought up aggressively prior to the pump. *quote_abs_zscore* indicates the same behaviour, this means that prior to the pump volumes are abnormal, which corresponds to insider trading volume. The remaining high-importance features are dominated by long-horizon trade-count and volume statistics (*num_trades* and *quote_abs_zscore* at the 2–14 day scale), which is consistent with manipulators selecting tokens with a characteristic liquidity and pre-pump accumulation profile rather than reacting to short-term price moves alone.

To verify that this is not an artifact of a single fit, we recomputed the feature importances inside each of the 25 random 70% training-subset retraining runs of Section IV-I, which yields a bootstrap distribution of importance for every feature. Table VII reports the per-feature percentiles of the 15 most important features, sorted by median. The set of leading features is stable across runs, with *num_prev_pump* attaining the highest median importance, confirming that the dominant predictors are a robust property of the model rather than of a particular training sample.

H. STATISTICAL SIGNIFICANCE

To quantify uncertainty around the observed performance differences, we employ cross-section-level bootstrap resampling. Each bootstrap sample draws 58 test-set cross-sections with replacement, and all metrics are recomputed on the resampled cross-sections. Resampling at the cross-section level (rather than the row level) is the methodologically appropriate choice here: each pump event is the natural unit of observation, and rows belonging to the same event are highly dependent because they share the cross-section-standardized denominator (Eq. 5). Row-level bootstrapping would produce optimistically narrow intervals.

Table VIII reports the Top@K%-AUC point estimate together with its 95% bootstrap confidence interval for Catboost + ES and the strongest static-metric baseline, CatboostClassifier + Tuned. The intervals are wide ($\pm \approx 0.08$), which reflects both the small number of positive test events ($n = 58$) and the fact that the new $(0, 20\%]$ integration range of Top@K%-AUC (Eq. 6) concentrates the metric on the steep, high-variance region of the Top@K% curve.

Fig. 8 visualizes the 95% bootstrap confidence intervals of Catboost + ES at each K and $K\%$. To test whether the improvement over the strongest baseline is real, we also conduct a *paired* bootstrap hypothesis test: on each of the 2000 resamples, both Catboost + ES and CatboostClassifier + Tuned are scored on the same resampled cross-sections, and we record the per-resample difference in Top@K%-AUC. The observed difference is 0.009 with paired 95% CI $[-0.017,\ 0.035]$ and a one-sided p-value of 0.250. The advantage of the TOPKAUC-early-stopped model is therefore in the expected direction but is *not* statistically significant at the 5% level on the paired test, so Tables V and VI should be read as a ranking of point estimates that remains consistent with the CatboostClassifier + Tuned baseline within sampling noise. The headline K-by-K advantage of TOPKAUC is most visible at $K = 10, 20$, and 30 (Table V), which is where the cumulative Top@K%-AUC separation is largest.

I. ROBUSTNESS ANALYSIS

1) Training data perturbation

We assess the stability of Catboost + ES by retraining it 25 times, each time using a random 70% subset of the training cross-sections while keeping the validation and test sets fixed. Table IX summarizes the distribution of test-set metrics across runs.

The Top@K%-AUC exhibits a standard deviation of only 0.017 across runs, with 80% of runs falling within $[0.683, 0.729]$. Fig. 9 visualizes the distribution. This demonstrates that performance is not driven by a fortunate training-set composition.

2) Temporal subperiod analysis

Our test set spans from May 2021 to March 2024, a period that includes the late 2021 bull market, the 2022 contagion (Terra/LUNA collapse in May 2022, FTX insolvency in November 2022), and the subsequent low-volume crypto winter of 2023. We conducted the subperiod analysis with the June 2022 split chosen *ex ante* as the conventional bull→bear regime boundary, so that the split is not a function of the model's test-set performance.

The early subperiod (May 2021–June 2022, 55 valid pump events) attains a test-set Top@K%-AUC of 0.737, essentially indistinguishable from the 0.729 figure on the full test sample. The late subperiod (July 2022–March 2024) contains only 3 labeled pump events, which reflects the post-2022 collapse in Telegram-organized pump activity on Binance, not a limitation of the model. We therefore apply a `min_pumps=10` guard and explicitly exclude the late subperiod from quantitative comparison: any Top@K%-AUC estimated on three events has a bootstrap 95% CI too wide to be useful. We acknowledge two implications: (i) within the part of the test set where the metric can be reliably computed, performance is stable across the 2021 bull-end and 2022 contagion regimes; (ii) generalizability to a deep, sustained bear market cannot be concluded from this test set alone and is flagged in Section VI. The fact that the model was trained exclusively on pre-September-2020 data and still delivers stable 2021–2022 out-of-sample performance is consistent with persistent microstructure signatures of pre-pump accumulation, rather than regime-specific overfitting.

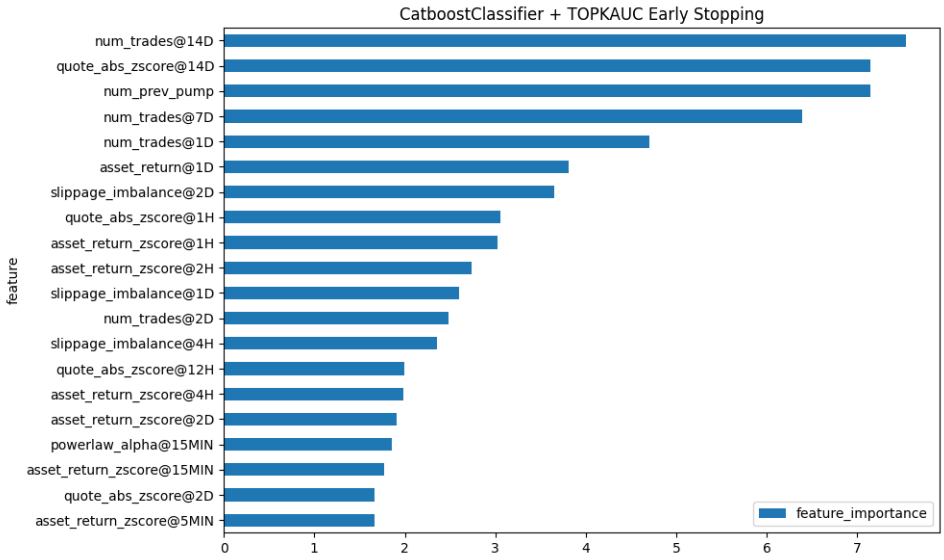


FIGURE 7. Catboost + ES Feature importances

TABLE VII. Bootstrap distribution of Catboost + ES feature importances (PredictionValuesChange) across the 25 random 70% training-subset retraining runs of Section IV-I. The 15 features with the highest median importance are shown; columns report per-feature percentiles and rows are sorted by median.

Feature	10%	25%	Median	75%	90%
num_prev_pump	5.95	7.65	9.86	11.23	13.19
quote_abs_zscore@14D	2.82	3.91	5.34	6.00	6.77
num_trades@14D	2.40	2.98	5.30	7.67	9.08
num_trades@2D	1.98	2.73	4.60	6.26	8.43
asset_return_zscore@2H	2.20	2.68	4.06	4.69	5.79
num_trades@7D	1.99	3.05	3.48	6.10	9.39
asset_return_zscore@4H	2.01	2.29	3.18	3.61	5.23
slippage_imbalance@2D	1.66	2.11	3.10	4.04	4.72
asset_return_zscore@1H	1.89	2.65	3.07	4.67	4.96
asset_return@1D	0.84	1.49	2.33	3.53	4.52
quote_abs_zscore@1H	1.13	1.96	2.30	3.22	3.81
asset_return_zscore@2D	0.50	0.67	2.27	3.32	4.40
num_trades@1D	1.12	1.42	1.93	2.81	4.58
quote_abs_zscore@12H	0.81	1.02	1.87	2.52	3.01
asset_return_zscore@15MIN	1.23	1.37	1.79	1.97	3.07

TABLE VIII. Top@K%-AUC on the test set with 95% cross-section-level bootstrap confidence intervals (1000 iterations). Top@K%-AUC is integrated and normalized over $K\% \in (0, 20\%]$ (Eq. 6).

Model	Top@K%-AUC	95% CI
CatboostClassifier + Tuned	0.720	[0.634, 0.794]
Catboost + ES	**0.729**	[**0**.644, **0**.807]

TABLE IX. Robustness analysis: distribution of test-set metrics across 25 runs with random 70% training subsets. Top@K%-AUC is integrated and normalized over $K\% \in (0, 20\%]$ (Eq. 6). The low standard deviation of Top@K%-AUC ($\sigma = 0.017$) indicates stable performance.

Metric	Mean	Std	[10%, 90%]
Top@K%-AUC	0.706	0.017	[0.683, 0.729]
Top@1%	0.299	0.037	[0.259, 0.345]
Top@2%	0.435	0.031	[0.397, 0.466]
Top@5%	0.623	0.034	[0.586, 0.666]
Top@10%	0.766	0.029	[0.724, 0.793]
Top@20%	0.896	0.031	[0.852, 0.931]

V. PORTFOLIO STRATEGY

In this section we will propose a portfolio based strategy similar to the one proposed by [17], but we will differ in the way portfolios are created. Authors of [17] proposed using trained classification models to produce logits for each asset in the cross-section, then construct a portfolio of assets that

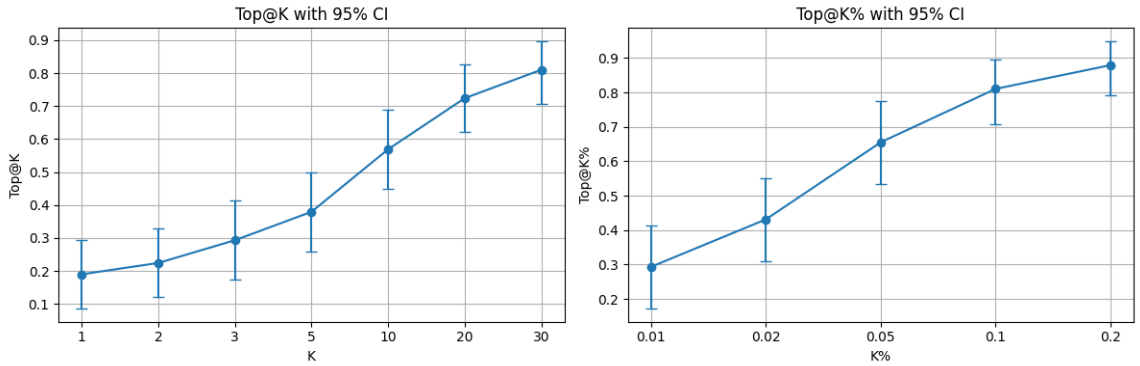


FIGURE 8. **Cross-section-level bootstrap 95% confidence intervals for Catboost + ES on the test set, based on 2000 resamples of the 58 test cross-sections with replacement. Left: Top@K accuracy with error bars at $K \in \{1, 2, 5, 10, 20, 30\}$. Right: Top@K% accuracy with error bars at $K\% \in \{1, 2, 5, 10, 20, 50\}$. The dashed horizontal segment in each panel is the random-ranking baseline at the same K / K%. Takeaway: all Top@K and Top@K% point estimates of the best model lie well above random even at the lower 2.5% bootstrap quantile, but interval widths are non-trivial and argue for cautious interpretation of narrow differences between models.**

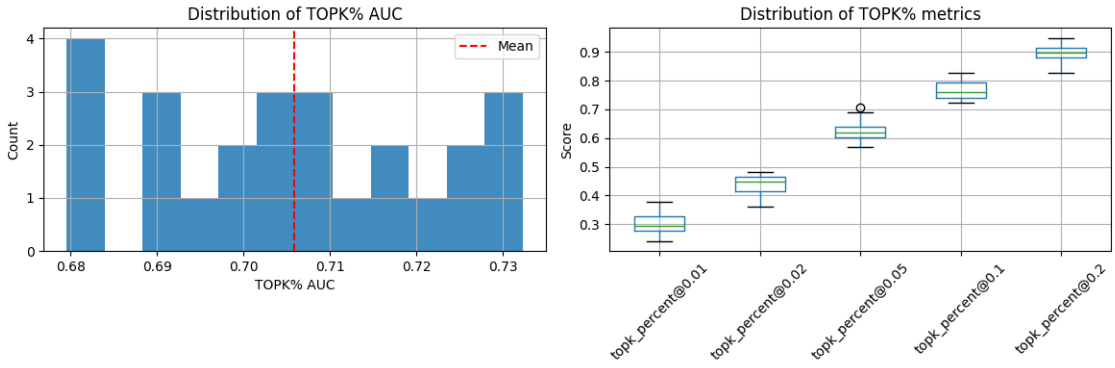


FIGURE 9. **Distribution of test-set metrics of Catboost + ES across 25 independent retraining runs. Each run uses a different random 70% subset of the training cross-sections; the validation and test sets are held fixed. Left: histogram of Top@K%-AUC (integrated and normalized over $K\% \in (0, 20\%]$) across the 25 runs; the vertical dashed line is the across-run mean. Right: box plots of Top@K% at $K\% \in \{1, 2, 5, 10, 20\}$. Takeaway: $\sigma_{\text{Top@K\%-AUC}} = 0.017$ and 80% of runs fall in [0.683, 0.729], so the headline performance is not driven by a fortunate training-set composition.**

have their logit of being pumped above some threshold. They also proposed using weights of the portfolio proportional to the logits produced by the model. They backtested their strategy using the following assumptions: assets that are not pumped do not change in price over the pump, authors buy at open price of the last hourly candle prior to the pump and are able to sell at half of pump's max return. They backtested their strategy using the test sample from October 29, 2018 to January 11, 2019 and were able to achieve a return of 60% in measly 2 months. We would like to further work on this approach and apply our trained models to create similar portfolios but we will deviate in our assumptions making them more realistic.

We propose using models tuned for Top@K%-AUC which will ensure that the models are optimized not only for Top@1 accuracy but for any arbitrary values of K. We relaxed some assumptions of [17] in the way we trade our portfolios, the timeline of how we trade each pump is described below:

1) Rank tickers by model logits (descending).
2) Select the top-k tickers.
3) Buy each selected ticker 15 minutes before the announcement with equal weights.
4) Hold until the announcement; if not pumped, sell at the announcement; if pumped, sell 1 minute after the pump.
5) Compute portfolio return minus 25 bp (0.25%) transaction cost.

We used the best performing model - CatboostClassifier + TOPKAUC Early Stopping as the primary model to create portfolios because it showed the superior results in terms of all evaluation metrics. We backtested the strategy using the steps described above for various portfolio sizes of K. Moments of portfolio return distributions are presented in Table X.

We can see that smaller portfolios tend to outperform larger portfolios in terms of mean returns; this is because we spread out our position across less assets, and therefore we are able to take advantage of correctly labeling the pumped ticker if it is in our portfolio. But this still increases the risk that we take on, as returns become more volatile and maximum drawdown also tends to increase with lower values of K. In Fig. 10 we can see all models compared to each other in terms of cumulative return over time. Return was calculated without reinvestment with fixed position size for all trades.

K	Avg. trade return	Annualized portfolio return	Annualized portfolio volatility	Sharpe ratio
1	0.0766	3.0801	1.1345	2.7149
2	0.0473	1.9019	0.6350	2.9952
5	0.0310	1.2477	0.3641	3.4266
10	0.0210	0.8449	0.2206	3.8298
20	0.0136	0.5455	0.1353	4.0306
30	0.0084	0.3392	0.0883	3.8430

TABLE X. Performance summary by portfolio size K

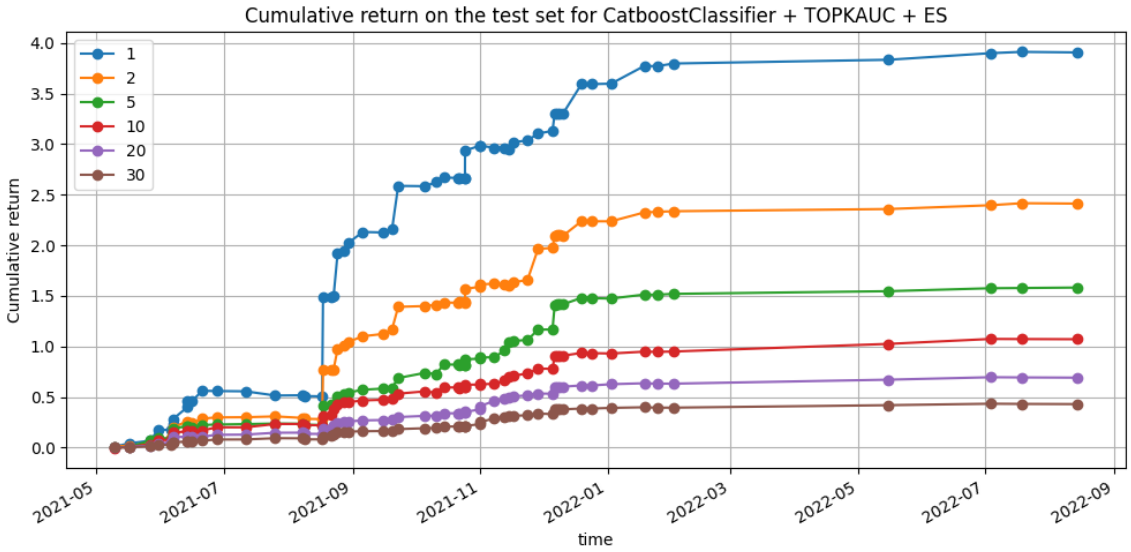


FIGURE 10. Cumulative return paths on the test set for portfolios of size $K \in \{1, 2, 5, 10, 20, 30\}$, ranked by Catboost + ES scores, without reinvestment and after a 25 bps round-trip cost.

Over the same event windows, a BTC buy-and-hold baseline delivers an annualized return of -0.431 with a Sharpe ratio of -0.662, indicating that the portfolio results are not explained by passive BTC exposure.

A. EXECUTION-AWARE BACKTESTING WITH PRICE IMPACT

The results above assume that orders can be filled at the prevailing mid-price, which is unrealistic for the illiquid small-cap tokens typically targeted by P&D operations. To assess the practical feasibility of the strategy, we extend the backtesting framework with a fitted market-impact model that produces *size-dependent, dynamic* slippage, rather than the flat fee implicit in prior work. The model serves two purposes: first, it estimates a realistic volume-weighted average price (VWAP) for both entry and exit; second, the shape of the impact curve is itself the *liquidity constraint* of the strategy—larger orders remain executable, but the fitted $\beta\sqrt{Q}$ curve makes each additional unit of notional arrive at progressively worse prices, so the economically relevant question shifts from "is the order executable?" to "at what order size does impact erode the signal?".

1) Impact model specification

We model price impact in basis points as a function of order notional Q (in USDT):

$$I(Q) = \beta\sqrt{Q}. \tag{8}$$

The square-root specification is a well-established empirical regularity in market microstructure. Kyle [26] introduced the foundational model of linear price impact from informed order flow; subsequent work demonstrated that the aggregate impact of executed orders follows a concave, approximately square-root, relationship with order size. Almgren and Chriss [20] formalized this within the optimal execution framework, while Almgren et al. [21] provided direct empirical estimates of the power-law exponent (≈ 0.6) using institutional equity data. Bouchaud et al. [23] surveyed the evidence for the square-root law across asset classes, and Tóth et al. [29] demonstrated its universality across stocks and time periods using a large meta-order dataset. In cryptocurrency markets specifically, Donier and Bonart [24] confirmed

the square-root law in Bitcoin using over one million reconstructed meta-orders, showing that the relationship holds across four decades of order sizes.

We fit a single coefficient $\beta \geq 0$ via constrained OLS (no intercept) for each asset, pooling absolute impacts from both buy and sell candles. The entry regression uses the previous 14 days of 5-minute candles constructed from trade-level data relative to the pump time, while the manipulated-asset exit regression uses 5-second sell-dominated candles from the first 10 minutes after the announcement. In each case, the absolute net volume serves as Q and the absolute close-to-close price change (in bps) serves as the impact observation. All notionals are converted to USDT using the indicative BTC/USDT rate at fitting time, so the impact curve remains comparable across BTC price regimes.

2) Impact measurement from candle data

Entry model (5-minute candles). We aggregate the 14-day pre-pump trade history into 5-minute candles. For each candle we compute the net buy volume $Q_{\text{net}} = 2 \times V_{\text{taker buy}} - V_{\text{total}}$ (in quote currency), where $V_{\text{taker buy}}$ is the taker-buy quote volume and V_{total} is the total quote volume. The absolute net volume $|Q_{\text{net}}|$ serves as the order size, and the absolute close-to-close price change $|p_{\text{close}}/p_{\text{prev close}} - 1| \times 10^4$ bps serves as the impact observation. Observations from both buy- and sell-dominated candles are pooled into a single regression, which doubles the effective sample size and produces a more stable estimate of β. The 5-minute resolution matches the intended execution horizon: we assume the order is worked over a single candle interval, so the candle-level impact directly estimates the cost of execution.

Exit model for manipulated assets (5-second candles, sell-only). For the pumped asset, we fit a separate model using 5-second candles from the first 10 minutes after the pump announcement. Only sell-dominated candles (negative Q_{net}) are used, because buying and selling are not balanced during the manipulation event and exit execution is sell-side only. The finer resolution captures the rapid liquidity dynamics during position liquidation.

We do not impose a hard volume cap. Any intended order size is assumed executable against the order book, while the fitted impact curve converts larger sizes into increasingly adverse VWAPs.

3) VWAP entry and exit price estimation

The execution simulation proceeds as follows for each selected asset:

Step 1: Determine raw prices. The entry price p^{entry} is the last trade price at or before $t_{\text{pump}} - \Delta t_{\text{buy}}$. The exit price p^{exit} is the first trade price at or after t_{pump} (or $t_{\text{pump}} + \Delta t_{\text{sell}}$ for the manipulated asset). These are the prices observable on the tape before execution.

Step 2: Resolve intended notional. The target order size Q_{intended} is specified in USDT and converted to quote currency (BTC) using the indicative BTC/USDT rate at entry time. This normalization ensures that intended position sizes are comparable across BTC price regimes.

Step 3: Set executed notional. In our execution model, the round-trip uses the full intended size on both legs:

$$Q_{\text{exec}} = Q_{\text{intended}}. \tag{9}$$

We assume that deeper layers of the order book remain available beyond the volume observed in historical candles. Larger trades therefore remain feasible, but they incur larger slippage through the impact model.

Step 4: Compute VWAP execution prices. The fitted coefficient estimates the price displacement associated with a given net order flow. An order that walks through the book continuously fills at improving prices along the way. Integrating the square-root impact profile over the execution path yields the average fill cost:

$$I_{\text{vwap}}(Q) = \frac{1}{Q}\int_0^Q \beta\sqrt{v}\,dv = \tfrac{2}{3}\,\beta\sqrt{Q}. \tag{10}$$

The VWAP impact is two-thirds of the terminal impact because early fills occur at better prices than the final fill level. The VWAP execution prices are then:

$$p^{\text{entry}}_{\text{vwap}} = p^{\text{entry}}\left(1 + I_{\text{vwap}}(Q_{\text{exec}})/10^4\right), \tag{11}$$

$$p^{\text{exit}}_{\text{vwap}} = p^{\text{exit}}\left(1 - I_{\text{vwap}}(Q_{\text{exec}})/10^4\right). \tag{12}$$

Buying pushes the fill price above the initial quote; selling pushes it below. The transaction return is

$$r = p^{\text{exit}}_{\text{vwap}}/p^{\text{entry}}_{\text{vwap}} - 1 - 0.0025, \tag{13}$$

where the 25 bps term covers the round-trip exchange fee.

Manipulated-asset exit. For the pumped asset the sell leg occurs during the manipulation window where buying and selling pressure are not balanced. A separate impact model is fitted on 5-second candles constructed from the first 10 minutes of trades after t_{pump}, using *only sell-dominated candles* (those with negative net buying volume). The finer 5-second resolution captures the rapid liquidity dynamics during position exit, and restricting to sell candles ensures the regression estimates the cost of liquidating a position rather than the mixed buy/sell regime of the pump itself. If insufficient sell-side data is available, the framework falls back to the pre-pump entry model.

4) Regression fit quality

Fig. 11 shows an example of the fitted entry impact regression for one pump asset. The absolute-impact approach pools buy and sell 5-minute candles into a single regression ($I = \beta\sqrt{Q}$, no intercept), which doubles the effective sample size and produces a more stable estimate ($R^2 = 0.324$ buy side, $R^2 = 0.355$ sell side, $\beta = 0.7316$ in the example shown).

Fig. 12 shows the corresponding exit impact regression for a manipulated asset. The sell-only 5-second candle approach yields $R^2 = 0.274$ with $\beta = 1.3203$ ($n = 83$ candles), confirming that the square-root specification captures impact dynamics even in the high-activity manipulation window. The

higher β reflects the increased cost of selling during a pump, consistent with the asymmetric liquidity conditions.

5) Unmodeled execution frictions

The fitted impact model captures average slippage against historical candle-level liquidity but intentionally omits several real-world frictions in order to keep the estimate interpretable. Specifically, we do not model (i) order-book depth exhaustion at the deepest levels (which would manifest as partial fills rather than a smooth $\beta\sqrt{Q}$ curve), (ii) queue priority and time-in-force dynamics on the exchange matching engine, (iii) exchange-side API latency between signal generation and order arrival, and (iv) the risk that other market participants detect the same pump signature and front-run the entry. Each of these frictions works in the same direction and would reduce realized PnL further, so the sensitivity numbers reported below should be read as an upper bound on achievable performance.

6) PnL sensitivity to order size

We sweep the intended order size from 100 to 10,000 USDT and re-run the impact-aware backtest for a $K = 5$ portfolio using a 60-minute pre-announcement entry and a 1-minute post-announcement exit. Fig. 13 reports the no-reinvestment cumulative ROE over the full 58-event test sample as a function of intended order size under the candle-based impact model. Cumulative ROE declines monotonically from 2.45 at 100 USDT to 1.65 at 10,000 USDT, so the strategy remains profitable across the tested range but loses a substantial part of its edge as size increases. This analysis demonstrates that while the strategy is viable for retail-scale execution, larger positions would require more sophisticated execution algorithms (e.g., time-weighted average price (TWAP)/VWAP splitting) to manage market impact. Scaling further to institutional notionals (100k–1M USDT) lies further down the same square-root impact curve and would erode the edge faster still, and more advanced policies—such as learning an optimal exit point instead of the fixed 1-minute exit used here—could improve realized returns; both directions require deeper order-book reconstruction than our candle-based fit supports and are left to future work.

VI. LIMITATIONS AND FUTURE WORK

Exchange generalizability. Our analysis is restricted to Binance BTC pairs. P&D dynamics may differ on smaller exchanges with lower liquidity, different fee structures, or weaker surveillance. Validating the framework on additional exchanges is a natural extension, though it requires comparable data availability.

Dataset size. With 359 labeled events (58 in the test set), statistical power is limited. The bootstrap confidence intervals (Section IV-H) quantify this uncertainty, but larger labeled datasets would enable more precise performance estimation.

Execution assumptions. Although we incorporate a fitted price-impact model estimated from pre-pump 5-minute candles and post-pump 5-second sell candles (Section V-A), the backtesting framework does not model all real-world frictions, including exchange API latency, order-book dynamics during the pump itself, or the risk of front-running by other participants who detect the same signal.

Evolving manipulation tactics. Manipulators may adapt their strategies over time. Our temporal subperiod analysis (Section IV-I) provides evidence of cross-regime stability, but ongoing retraining and monitoring would be necessary for deployment.

Alternative imbalance-handling approaches. While we compared class weighting and SMOTE, other techniques such as focal loss [27], cost-sensitive learning [25], and deep anomaly detection [28] remain unexplored in this setting and could potentially improve performance.

VII. CONCLUSION

In this study, we successfully applied machine learning techniques to predict pump-and-dump targets, achieving state-of-the-art performance with our formulated approaches. Our analysis demonstrates that identifying P&D targets in advance is not only feasible but also presents opportunities for profitable trading strategies. Specifically, we found that a CatboostClassifier, combined with an imbalance-aware training procedure, yielded the best results and could be transformed into a portfolio-based trading strategy.

Our findings also explored alternative classification methods on imbalanced data, including ranking algorithms. However, these approaches demonstrated inferior performance when compared to boosting algorithms, such as CatboostClassifier. Our research has significant implications for market surveillance and regulatory policy, highlighting the potential benefits of using machine learning techniques to detect and prevent pump-and-dump schemes.

REFERENCES

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019.

[2] J. Albers, M. Cucuringu, S. Howison, and A. Y. Shestopaloff, "Fragmentation, price formation and cross-impact in Bitcoin markets," *Appl. Math. Finance*, vol. 28, 2022.

[3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, 2001.

[4] V. Chadalapaka, K. Chang, G. Mahajan, and A. Vasil, "Crypto pump and dump detection via deep learning techniques," *arXiv*, 2022.

[5] L. W. Cong, X. Li, K. Tang, and Y. Yang, "Crypto wash trading," *Manage. Sci.*, vol. 69, no. 11, 2023.

[6] D. Fantazzini and Y. Xiao, "Detecting pump-and-dumps with crypto-assets: Dealing with imbalanced datasets and insiders' anticipated purchases," *Econometrics*, vol. 11, no. 3, 2023.

[7] L. Grinsztajn, O. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems*, 2022.

[8] S. Hu, Z. Zhang, S. Lu, B. He, and Z. Li, "Sequence-based target coin prediction for cryptocurrency pump-and-dump," *Proc. ACM Manag. Data*, vol. 1, no. 1, 2023.

[9] J. Kamps and K. Bennett, "To the moon: Defining and detecting cryptocurrency pump-and-dumps," *Crime Sci.*, vol. 7, p. 18, 2018.

[10] M. La Morgia, A. Mei, F. Sassi, and J. Stefa, "Pump and dumps in the Bitcoin era: Real-time detection of cryptocurrency market manipulations," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2020, pp. 1–9.

[11] M. La Morgia, A. Mei, F. Sassi, and J. Stefa, "The doge of wall street: Analysis and detection of pump-and-dump cryptocurrency manipulations," *ACM Trans. Internet Technol.*, vol. 23, no. 1, pp. 1–28, 2023.

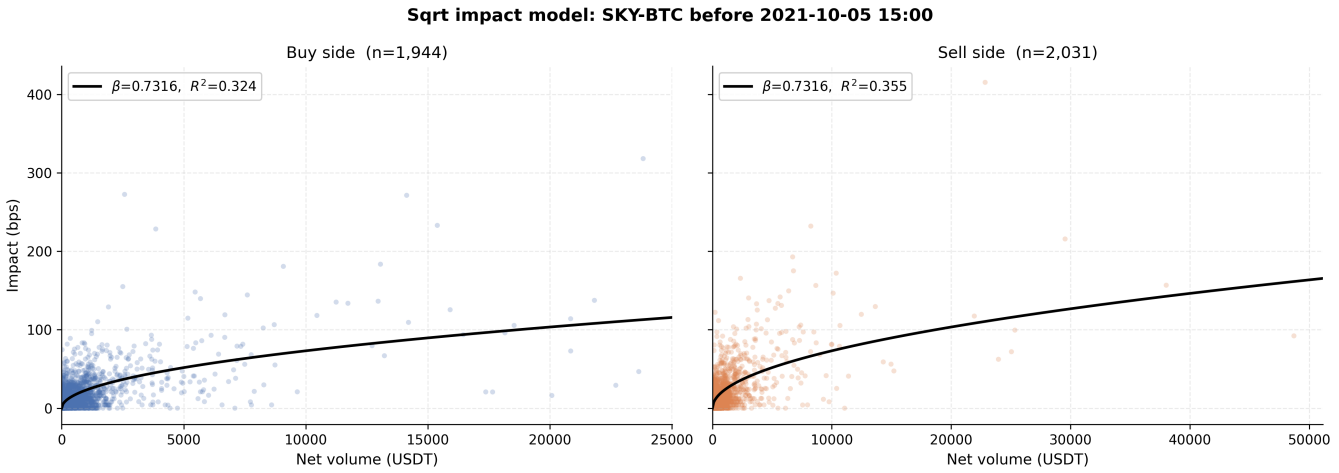


FIGURE 11. Fitted entry price-impact regression for the SKY-BTC pair, using the 14 trading days of 5-minute candles preceding the pump announcement. Left panel: candles with net buying pressure (positive Q_{net}); right panel: candles with net selling pressure (negative Q_{net}). Each scatter point is one 5-minute candle; horizontal axis $|Q_{\text{net}}|$ is absolute net volume in USDT and vertical axis $|\acute{p}|$ is the absolute close-to-close price change in basis points. Solid line: fitted $|\acute{p}| = \beta\sqrt{|Q_{\text{net}}|}$ (no intercept, $\beta = 0.73$), estimated on both panels jointly. **Takeaway:** the single-parameter square-root fit captures the concave impact–size relation on both sides of the book, consistent with the square-root impact law documented across asset classes [24], [29]; this is the curve used to translate order size into VWAP slippage in the backtest.

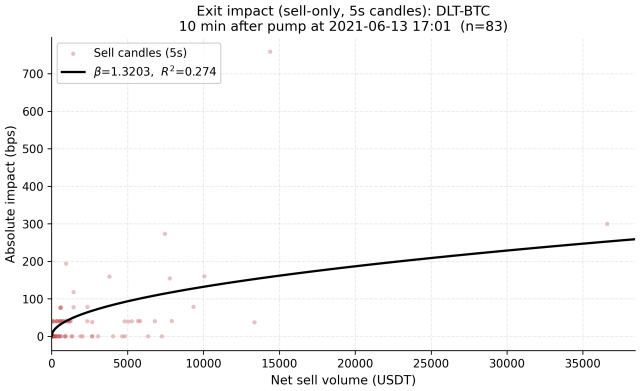


FIGURE 12. Fitted exit price-impact regression for the DLT-BTC pair, using 5-second candles from the first 10 minutes after the pump announcement and restricted to sell-dominated candles ($n = 83$). Horizontal axis: absolute sell-side net notional $|Q_{\text{net}}|$ in USDT. Vertical axis: absolute close-to-close price change in basis points. Solid line: fitted $|\acute{p}| = \beta\sqrt{|Q_{\text{net}}|}$ ($\beta = 1.32$). **Takeaway:** the exit coefficient is nearly twice the entry coefficient (1.32 vs. 0.73 on the earlier example), which reflects the asymmetric liquidity conditions during a pump—selling into the dying pump is materially more expensive than buying against resting depth pre-announcement.

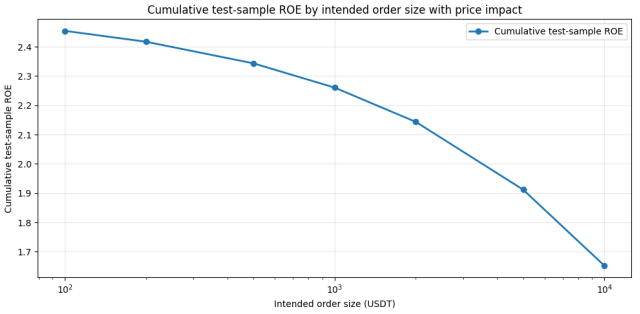
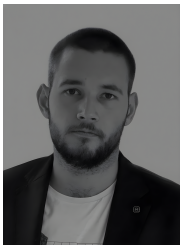


FIGURE 13. Cumulative return on invested capital (ROE) on the test sample as a function of intended per-trade order size, with the fitted candle-based square-root impact model enabled on both legs. Horizontal axis (log scale): intended order size in USDT, swept from 100 to 10,000. Vertical axis: cumulative ROE across the 58 test-set events for a $K = 5$ portfolio with a 60-minute pre-announcement entry and 1-minute post-announcement exit. **Takeaway:** cumulative ROE falls monotonically from 2.45 at 100 USDT per trade to 1.65 at 10,000 USDT per trade, so the strategy remains profitable across the tested range but loses a substantial share of its edge as size increases; at notionals much beyond this range, execution algorithms (e.g. TWAP/VWAP splitting) would be required.

[12] H. Nghiem, G. Muric, F. Morstatter, and E. Ferrara, "Detecting cryptocurrency pump-and-dump frauds using market and social signals," *Expert Syst. Appl.*, art. no. 115284, 2021.
[13] A. Ntakaris, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Mid-price prediction based on machine learning methods with technical and quantitative indicators," *PLOS ONE*, vol. 15, no. 6, 2020.
[14] D. Poh, B. Lim, S. Zohren, and S. Roberts, "Building cross-sectional systematic strategies by learning to rank," *J. Financial Data Sci.*, vol. 3, no. 2, pp. 70–86, 2021.
[15] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.
[16] F. Victor and T. Hagemann, "Cryptocurrency pump and dump schemes: Quantification and detection," in *Proc. Int. Conf. Data Mining Workshops (ICDMW)*, 2019.
[17] J. Xu and B. Livshits, "The anatomy of a cryptocurrency pump-and-dump scheme," in *Proc. 28th USENIX Secur. Symp. (USENIX Security 19)*, 2019, pp. 1609–1625.
[18] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
[19] M. Mironov, D. Bogutsky, and P. Lukianchenko, "Pump-and-dump target prediction (code repository), GitHub, 2025. [Online]. Available: https://github.com/B0R0koko/pump-and-dump-prediction. Accessed on: Dec. 24, 2025.
[20] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *J. Risk*, vol. 3, no. 2, pp. 5–39, 2001.
[21] R. Almgren, C. Thum, E. Hauptmann, and H. Li, "Direct estimation of equity market impact," *Risk*, vol. 18, no. 7, pp. 58–62, 2005.
[22] R. Blagus and L. Lusa, "SMOTE for high-dimensional class-imbalanced data," *BMC Bioinform.*, vol. 14, p. 106, 2013.
[23] J.-P. Bouchaud, J. D. Farmer, and F. Lillo, "How markets slowly digest changes in supply and demand," in *Handbook of Financial Markets:*

Dynamics and Evolution, T. Hens and K. R. Schenk-Hoppé, Eds. Elsevier, 2009, ch. 2.
[24] J. Donier and J. Bonart, "A million metaorder analysis of market impact on the Bitcoin," *Market Microstructure and Liquidity*, vol. 1, no. 2, art. 1550008, 2015.
[25] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2001, pp. 973–978.
[26] A. S. Kyle, "Continuous auctions and insider trading," *Econometrica*, vol. 53, no. 6, pp. 1315–1335, 1985.
[27] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.
[28] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
[29] B. Tóth, Y. Lemperiere, C. Deremble, J. de Lataillade, J. Kockelkoren, and J.-P. Bouchaud, "Anomalous price impact and the critical nature of liquidity in financial markets," *Phys. Rev. X*, vol. 1, art. 021006, 2011.



MIKHAIL MIRONOV received the B.S. degree in economics from Universitat Pompeu Fabra, Barcelona, Spain, and the National Research University Higher School of Economics (HSE), St. Petersburg, Russia, in 2023 (double degree), and the M.S. degree with honors in financial economics from International College of Economics and Finance, ICEF HSE, Moscow, Russia, in 2025. He is currently pursuing the Ph.D. degree in finance at HSE, Moscow, Russia. His major field of study is finance and financial economics.

DENIS BOGUTSKY received the M.S. degree in mathematics from Lomonosov Moscow State University in 2020. He is currently a Junior Researcher with the Laboratory of AI in Mathematical Finance at the National Research University Higher School of Economics. His research focuses on quantitative finance, market microstructure, credit risk modeling, and anomaly detection in financial systems.

PETR LUKIANCHENKO received the Bachelor's and Master's degrees in information science from HSE University, Moscow, Russian Federation, in 2009 and 2011, respectively, then received PhD in Math as well.

He has been with HSE University since 2009. He currently holds multiple positions there, including Senior Lecturer with the Faculty of Computer Science, Big Data and Information Retrieval School, Lecturer with the Centre for Training Top AI Professionals; and Head of the Laboratory of Artificial Intelligence in Mathematical Finance with the AI and Digital Science Institute. His responsibilities include leading research projects on the application of artificial intelligence in financial mathematics. His research focuses on AI applications in finance, including the detection of structural breaks in financial time series and the development of simulators for financial market crises. He has co-authored research on neural network approaches for predicting anomalies in interest rates under stochastic noise. He presented his work on developing a multi-agent simulator to recreate crisis events in financial markets at the international AI Journey 2025 conference.